Compilation

The phases of compilation are grouped into three stages that are usually executed in three passes through the preprocessed source code:

- **front end** that is responsible for the analysis of source code
- **middle end** that is responsible for performing optimizations of the code
- **back end** (or code generator) that is responsible for the production of the target code

Front-End Compiler Structure

The front end is responsible for analyzing the source code for syntax, semantics, and other tasks. The first part of this process is **lexical analysis**, which converts the source code strings of characters into **tokens**. These are sequences of compiler recognizable atomic symbols which make it much easier for the next steps to recognize them.

The process proceeds to **syntax analysis**, usually referred to as **parsing** or reading the code to make sure it conforms with the syntax rules of the language. This is done to test conformability logically by breaking code into separate parts.

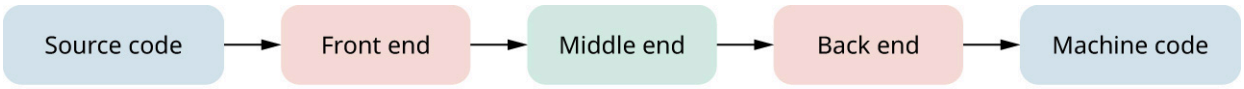
The next step is **semantic analysis**, which is the discovery of meaning in the code. It takes care of type checking in strongly typed languages. It also performs binding, which associates variable and function references with their declarations and definitions. The compiler runs a static semantic analysis which concerns itself only with the source code without testing inputs.

Semantic analysis cannot figure out all meaning in the code. Many languages also use dynamic binding (late binding) that defers the binding of certain elements such as objects and methods until runtime. This is particularly true of OOP languages which support inheritance and polymorphism. Some processes, such as when an array goes out of bounds, will not be known until run time, and are part of the program's dynamic semantics.

Semantic analysis concludes with the creation and management of the symbol table, a data structure that is used to connect every symbol with necessary information such as data type, scope, and memory location.

Middle-End Compiler Structure

The main job of the middle-end stage of compilation is **optimization**, producing what will be needed to generate the fastest and most efficiently running code as possible. The attributes which are usually optimized are execution time and space (memory usage). Figure 7.24 outlines this process.



Middle End	Task	Goal
	Optimization	Improve code
	Transforms	Analyze and rewrite code to reduce running time, space, and power consumption of the compiled code.

Figure 7.24 Code optimization is the phase between the front end and back end of the compilation process. (attribution: Copyright Rice University, OpenStax, under CC BY 4.0 license)

Intermediate Form

The result created after semantic analysis when the program passes all checks is called **intermediate form (IF)**. The nature of the IF is often chosen for machine independence and ease of optimization. IFs usually resemble the machine code for some imaginary idealized machine. An example is **managed code** which is targeted to run under a particular runtime environment. This is true of C# and other Microsoft.net languages. They are designed to run under its CLR, which manages the execution of Microsoft's .NET languages. In this